

2026-03-26 Robot Dog GitHub Reference Research

Executive Summary

本轮只针对“机器狗/四足机器人”步态与 IK 参考做筛选，不再把普通碳基四足生物动画和机器狗混在一起。结论很明确：对当前仓库这只机械狗，最值得借鉴的是“机器人式 gait 架构”，也就是 `body controller + gait scheduler + foothold planner + leg IK` 这条链，而不是直接照搬生物动画里那种骨骼驱动或整条腿跟随样条的做法 [1][3][4][5][6]。

结合本地现状，当前 `CityRobotDog.tscn` 只有 8 个 author 的 `hip/knee` 锚点，没有骨骼 rig，也没有 `foot/ankle` 锚点；前后腿上段长度还不完全相同。因此，最稳的第一版不是上 Godot 骨骼 IK 插件，而是先在 lab 里做“机器人式脚点规划 + 两段腿可视求解”，把走路问题拆开解决。

Key Findings

- 机器狗的主线不是“播动作”，而是“先解脚点，再解腿”：`StanfordQuadruped`、`spot_mini_mini`、`spotMicro` 都把系统拆成 `body posture`、`gait schedule`、`swing/stance leg controller`、`IK` 这些层，说明这是稳定主流做法 [3][4][5]。
- Godot 侧有可借鉴的程序步态 demo，但更像“落地风格参考”，不是直接可抄的机器狗 runtime：`TheBogdichHD/procedural-animation-godot` 展示了 Godot 中“目标脚点 + 成对迈步 + IK effectors”的组织方式，很适合作为引擎落地参考 [1]。
- 你当前模型不适合直接走骨骼 IK 插件路线：`GodotIK` 适合骨骼/链式 IK，但你现在这只机械狗是拆件模型 + `scene-first` 锚点，不是标准骨架绑定资产，所以它最多是后备参考，不是第一选择 [2]。
- 最适合你当前阶段的 gait 不是复杂 `trot controller`，而是先做低速 `crawl` 或保守 `trot`：机器人项目普遍支持 `walk/trot` 等多 gait，但第一版应先冻结可调试、可验证的低速 gait，而不是一口气冲动态平衡 [3][4][5][6]。

Detailed Analysis

本地现状与实现约束

本地 `v59` 已经完成的是 `scene/lab` 基础设施，而不是 `locomotion`。当前 `CityRobotDog` 暴露了 `BodyPivot`、`Model`、`JointAnchors` 与 8 个固定命名的关节锚点：`lf/rf/lr/rr` 的 `hip + knee`。

从锚点几何看，这只机械狗存在几个很关键的约束：

- 它没有 `foot` 锚点，说明脚底默认落点不能直接从 `authoring` 读出，必须由 `runtime` 自己维护“期望脚点”。
- 前腿 `hip -> knee` 段长约 `0.24m ~ 0.25m`，后腿约 `0.31m ~ 0.34m`，前后腿并不完全对称。
- 模型是拆件 `scene`，不是标准骨骼 `rig`；这意味着“腿的视觉姿态”更像是通过节点旋转/关节平面求解出来，而不是让动画系统直接接管。

这几点直接决定了：当前最稳的方向是“机器人式脚点求解 + `scene` 节点姿态同步”，而不是先重做骨架或引入复杂动画管线。

参考 1：Godot 程序四足步态落地风格

`TheBogdichHD/procedural-animation-godot` 是这轮里最贴近 Godot 落地的参考。它的 README 直接说明：通过检测腿是否离目标太远，再让对应的 `StepTargetContainer` 做一步，并把 IK 目标应用到腿链上 [1]。这和你现在要做的“脚点规划先行”非常接近。

它的价值不在于“机器狗数学绝对正确”，而在于它告诉你在 Godot 里这件事应该怎么拆：

- 每条腿有自己的脚点目标；
- `gait controller` 负责决定哪条腿该迈步；
- 步态不是所有腿一起动，而是按配对/交替规则动；
- 最终视觉腿姿态跟着脚点走，而不是反过来。

这条参考最适合借的是：Godot 工程组织方式、每条腿的 `stepping` 状态机、脚点目标节点管理。不适合直接照抄的是：它默认骨骼/IK `effectors` 语境更强，而你这里是拆件机械腿。

参考 2：GodotIK 适合作为后备，不适合作为第一版主线

`monxa/GodotIK` 提供的是通用链式 IK 能力，README 明确写了支持多个 `chain`、可脚本约束，并可在编辑器内可视化调试 [2]。如果你以后把机械狗重做成骨骼 `rig`，它会很有吸引力。

但对当前资产状态，它的问题也很明确：

- 你现在没有现成 skeleton/bone 链；
- 你当前的 authoring 真源是 8 个 Marker3D ；
- 你需要先保证“脚点、步态、身体高度、debug contract”这些机器人链路成立，而不是先把求解绑死在某个 IK 插件 API 上。

所以它更像后续扩展路线：

- 第一版：自己做 quadruped runtime，scene 节点驱动。
- 未来如果换 skeleton 版模型：再评估是否接 GodotIK 。

参考 3：SpotMicro 是“产品形态”参考

mike4192/spotMicro 非常像“完整开源机器狗产品栈”。README 明确列出了站立、平移、俯仰滚转偏航，以及 8 phase walking gait 和 trotting gait [3]。这说明机器狗 runtime 的输入通常不是“播某条腿动作”，而是：

- 身体目标高度/姿态；
- 前进、侧移、转向命令；
- gait 模式切换；
- gait 内部再去推导每条腿的脚点轨迹。

对你当前项目，它最值得借的是“上层控制语义”：

- 玩家或 AI 不该直接操某条腿；
- 机械狗 runtime 自己维护 gait phase；
- 每帧真正解的是“身体命令 -> 脚点 -> 关节姿态”。

参考 4：StanfordQuadruped 的架构拆分最值得抄

StanfordQuadruped 的仓库结构非常有参考价值。README 与源码目录把控制器拆成了 Controller、Gaits、StanceController、SwingLegController、InverseKinematics、State 等模块 [4]。这几乎就是你后续 CityRobotDog runtime 的理想分层。

这里最重要的不是它的参数，而是它的边界：

- Gaits 只负责 phase 与时序；
- SwingLegController 只负责抬腿时的轨迹；
- StanceController 只负责支撑腿与机身运动关系；
- InverseKinematics 只负责从脚点求关节。

这一点和你仓库的设计哲学是高度一致的：分层即道，协议优于实现。

参考 5: spot_mini_mini 说明了“机器人步态”该长什么样

`OpenQuadruped/spot_mini_mini` 是一个相对完整的开源机器狗实现，README 直接列出 forward/inverse kinematics、multiple gaits、PID control、gait scheduler、trajectory shaping 等核心模块 [5]。这说明机器狗和普通四足生物动画最大的差别在于：

- 机器人 gait 强依赖“调度器”和“轨迹塑形”；
- 脚的 swing 轨迹常常是被显式设计出来的；
- 支撑相和摆动相是明确区分的。

所以你说“机器狗和碳基四足不一样”，这个判断是对的。真正的机器狗实现，核心不是“更像动物”，而是“可控、可调、可验证”。

参考 6: Quadrupedal-Robot-Gait-Visualization 适合借数学直觉

`zainkhan-afk/Quadrupedal-Robot-Gait-Visualization` 虽然更偏可视化验证，但 README 直接写了用逆运动学与 `half ellipse motion` 来表达不同 gait [6]。这个仓库很适合作为“脚点轨迹曲线”的直觉参考：

- 机器人抬腿时，用半椭圆、Bezier、样条都可以；
- 第一版不要追求极致仿真，先把“支撑稳、迈步清楚、不会滑脚”搞定；
- 轨迹曲线以后再换，不应该影响 gait scheduler 和 IK 层。

Areas of Consensus

- 机器狗 locomotion 的稳定架构，普遍是 `body control -> gait phase -> foot target -> IK` 的分层 [3][4][5]。
- 支撑相和摆动相必须分开建模，不能所有腿同逻辑处理 [1][4][5][6]。
- 脚点通常需要显式规划，而不是单纯依赖播放动作或骨骼动画 [1][3][4][5]。
- 第一版更应该先把 walk/crawl/trot 的最小闭环跑稳，再谈更复杂的动态能力 [3][5][6]。

Areas of Debate

- 第一版用 **crawl** 还是 **trot**：crawl 更稳、更容易 debug；trot 更像机器狗，但调参更敏感 [3][4][5]。
- 是否一开始就接通用 **IK 插件**：插件接入更快，但会把当前 scene-first 拆件模型绑进插件语义；手写求解更稳但工作量更大 [2]。
- 脚点轨迹用半椭圆、Bezier 还是分段曲线：工程上差别不大，关键是轨迹层要和 gait 层解耦 [1][5][6]。

Recommendation For This Repo

推荐你当前仓库按下面这条线推进：

1. 只在 `RobotDogLab` 做第一版 locomotion，不接主世界。
2. 新建 quadruped runtime，而不是硬套 arthropod runtime。
3. runtime 分四层：
 - `gait scheduler`
 - `foothold / swing planner`
 - `body pose solver`
 - `leg visual solver`
4. 每条腿维护自己的：
 - `phase`
 - `mode ( stance/swing )`
 - `locked_foothold`
 - `desired_foothold`
 - `last_step_time`
5. 第一版 gait 先做保守 crawl 或低速 trot。
6. 第一版不要引入 GodotIK，不要重做 rig，不要做动态跳跃。

如果只问“哪条参考最值得先抄”：

- **架构最值得抄：** `StanfordQuadruped` [4]
- **Godot 落地风格最值得抄：** `procedural-animation-godot` [1]
- **机器人产品语义最值得抄：** `spotMicro` / `spot_mini_mini` [3][5]
- **脚点曲线最值得抄：** `Quadrupedal-Robot-Gait-Visualization` [6]

Sources

[1] TheBogdichHD, `procedural-animation-godot`, GitHub repository. Godot 四足程序动画 demo，展示脚点目标、迈步触发与 IK effectors 的基本组织方式。

<https://github.com/TheBogdichHD/procedural-animation-godot>

[2] monxa, `GodotIK`, GitHub repository. Godot 通用 IK 插件，支持多个 chain、脚本约束和可视化调试，适合骨骼链而非当前拆件 scene-first 资产。

<https://github.com/monxa/GodotIK>

[3] mike4192, `spotMicro`, GitHub repository. 开源机器狗项目, README 明确包含站立、姿态调整、`8 phase walking gait`、`trotting gait` 等控制语义。

<https://github.com/mike4192/spotMicro>

[4] stanfordroboticsclub, `StanfordQuadruped`, GitHub repository. 开源四足机器人控制栈, 目录拆分为 `gait`、`stance`、`swing`、`IK`、`state` 等模块, 分层非常清晰。

<https://github.com/stanfordroboticsclub/StanfordQuadruped>

[5] OpenQuadruped, `spot_mini_mini`, GitHub repository. 开源机器狗实现, 涵盖 `forward/inverse kinematics`、`multiple gaits`、`gait scheduler` 与 `trajectory shaping`。

https://github.com/OpenQuadruped/spot_mini_mini

[6] zainkhan-afk, `Quadrupedal-Robot-Gait-Visualization`, GitHub repository. 机器人四足 `gait` 可视化项目, 使用逆运动学与 `half-ellipse` 脚点轨迹说明不同 `gait`。

<https://github.com/zainkhan-afk/Quadrupedal-Robot-Gait-Visualization>

Gaps and Further Research

- 当前还没针对你这只机械狗的具体拆件层级做 runtime 级 joint mapping, 这一步需要进 Godot scene 看每个腿段节点名和局部轴向。
- 当前还没决定第一版 `gait` 是 `crawl` 还是 `low-speed trot`, 这需要你在“更稳”和“更像机器狗”之间做一次冻结。
- 当前还没把研究结论冻结成 `v60` 的正式计划与 DoD; 如果继续实施, 下一步应进入 `v60` docs freeze + TDD red。